

Jetson 端工业缺陷检测的输入数据路径重构

王凯伦, 王琦

合肥工业大学数学学院, 安徽 合肥 230601

通信作者: 王琦

摘要: 针对 Jetson 端工业缺陷检测中网络低精度化不能直接消除输入形成与主机—设备数据移动开销的问题, 本文在固定 YOLOv8n 和 TensorRT INT8 混合精度 Engine 的条件下, 将 CPU/OpenCV 侧形成 FP32 NCHW 张量并复制至设备的基线路径, 重构为 packed BGR 原始图像暂存、二维主机到设备复制以及 GPU 侧融合 CUDA 预处理直接形成 TensorRT 设备输入的路径。在统一的任务正确性、逐帧 E2E 延迟、进程级 FPS 和合并样本 P95/P99 评价口径下, 实验通过 V0→V2R 与 V2R→V3R 两级受控比较, 分别考察完整输入形成路径重构和 pageable→pinned 主机暂存变化。三条路径在冻结的 180 幅测试集上获得一致的任务级指标。V2R 相对 V0 达到 2.24× FPS, 平均延迟降低 55.45%, 构成主要 E2E 性能增量; V3R 相对 V2R 的 FPS 变化为 +4.07%, 平均延迟变化为 -4.03%, P95 变化为 +0.15%、P99 变化为 -0.12%, 两个尾延迟指标变化均低于 0.2%且方向相反。结果表明, 在本文固定 Jetson 平台、实现和离线回放工作负载下, 完整输入形成与数据移动路径重构带来主要平均性能收益, pinned 暂存只提供有限的平均增量, 尚无一致的尾延迟改善证据。

关键词: Jetson; 工业缺陷检测; INT8 混合精度推理; CUDA 预处理; 主机—设备数据路径

Input Data-Path Reconstruction for Industrial Defect Detection on Jetson

WANG Kailun, WANG Qi

School of Mathematics, Hefei University of Technology, Hefei 230601, China

Corresponding author: WANG Qi

Abstract: For Jetson-based industrial defect detection, reducing network precision does not by itself remove the costs of input formation and host-device data movement. With a fixed YOLOv8n detector and TensorRT INT8 mixed-precision Engine, this study restructures a baseline path that forms an FP32 NCHW tensor through CPU/OpenCV and copies it to the device into a path based on packed BGR raw-image staging, two-dimensional host-to-device copying, and fused CUDA preprocessing that directly forms the TensorRT device input. A unified protocol evaluates task correctness, source-to-pre-sink per-frame latency, process-level FPS, and pooled P95/P99 through two controlled comparisons: V0→V2R for the complete input-formation restructuring and V2R→V3R for pageable-to-pinned host staging. All three paths produce identical task-level metrics on the frozen 180-image test set. Relative to V0, V2R achieves 2.24× FPS and 55.45% lower mean latency, representing the principal end-to-end performance gain. Relative to V2R, the V3R changes are +4.07% in FPS and -4.03% in mean latency; the P95 and P99 changes are +0.15% and -0.12%, respectively, with both tail changes below 0.2% and in opposite directions. Under the fixed Jetson platform, implementation, and offline replay workload evaluated here, the complete input-formation and data-movement restructuring provides the main average-performance gain, whereas pinned staging adds only a limited mean improvement and does not provide evidence of consistent tail-latency improvement.

Keywords: Jetson; industrial defect detection; INT8 mixed-precision inference; CUDA preprocessing; host-device data path

中图分类号: TP391.41

0 引言

工业产品表面缺陷会影响产品质量与后续制造过程, 金属表面缺陷自动检测已成为工业视觉的重要应用方向^[1]。东北大学钢材表面缺陷数据库包含开裂、夹杂、斑块、点蚀表面、轧制氧化皮和划痕等典型缺陷, 为工业缺陷检测研究提供了常用实验基础^[2]。

现有表面缺陷检测研究多围绕检测器结构展开, 通过轻量化骨干、注意力或多尺度特征融合改善缺陷表征, 并讨论检测精度与模型复杂度之

间的权衡^[3-5]。YOLOv8 模型系列提供多种规模的检测模型以及训练、推理和导出能力^[6]。当检测器转移到资源受限的边缘平台后, 研究对象还包括模型之外的部署执行过程。已有工作将边缘 DNN 运行时作为独立系统问题, 并把预处理、推理和后处理纳入完整检测路径^[7]; 其他 AI 工作负载中的研究也表明, CPU 预处理及其数据管理可能限制加速器利用率和端到端 (end-to-end, E2E) 性能^[8]。工业现场附近的边缘部署则

王凯伦 (1999—), 男, 山东潍坊人, 工学学士, 硕士研究生, 主要研究方向为端侧人工智能推理部署与优化。

进一步要求系统在受限计算条件下完成完整处理流程^[9]。

TensorRT INT8 混合精度部署可减少网络侧计算开销，后训练量化能够在既有模型基础上实施低比特近似，但量化扰动仍需通过任务正确性评价加以约束^[10,11]。本文采用 TensorRT 10.3 中的校准式 INT8 路线作为固定推理前提；该版本的传统隐式 INT8 量化和 calibrator 接口已被标记为弃用，因而实验结论限定于本文的软件栈^[12]。网络低精度化并不会自动重组图像解码后的主机侧数据表示、输入张量形成位置或主机—设备数据移动，完整 E2E 性能仍取决于这些阶段在实际部署路径中的组织方式。

现有相关工作分别从检测模型、量化与推理、边缘运行时、GPU 预处理或系统级部署等角度讨论边缘检测性能。为减少模型和推理后端差异带来的混杂，本文进一步将研究范围收敛到固定检测器与 Engine 条件下的输入形成位置、主机侧数据表示、输入复制载荷以及可分页（pageable）/锁页（pinned）暂存，并通过两级受控路径比较评价其完整 E2E 行为。本文据此将研究重点放在 Jetson TensorRT INT8 输入形成与主机—设备数据路径工程：基线路径 V0 在主机

侧通过 CPU/OpenCV 形成 FP32 NCHW 张量并复制至设备；V2R 将其重构为 packed BGR 原始图像暂存、通过 cudaMemcpy2DAsync 执行二维 H2D 复制和 GPU 侧融合 CUDA 预处理，直接形成 TensorRT 设备输入；V3R 保持相同 CUDA 预处理、CUDA stream 和下游拓扑，仅将 pageable 原始图像暂存替换为 pinned 暂存。

本文的主要贡献包括两点：1）在固定 YOLOv8n 和 TensorRT INT8 混合精度 Engine 的条件下，将 CPU/OpenCV 主机侧 FP32 张量输入形成路径重构为 packed BGR 原始图像暂存、cudaMemcpy2DAsync 和融合 CUDA 预处理，使 CUDA 核函数直接形成 TensorRT 管理的 FP32 NCHW 设备输入，并通过 pageable/pinned 配置隔离主机暂存内存类型；2）在统一的任务正确性、E2E 延迟、进程级 FPS 与合并样本 P95/P99 评价口径下，通过 V0→V2R 和 V2R→V3R 两级受控比较，区分完整输入路径重构的主要性能收益与 pinned 暂存的有限平均增量，并利用 5 次独立进程考察运行级分布和尾延迟行为。

两级受控路径及其完整 E2E 观测关系见图 1。

受控输入数据路径工程总览

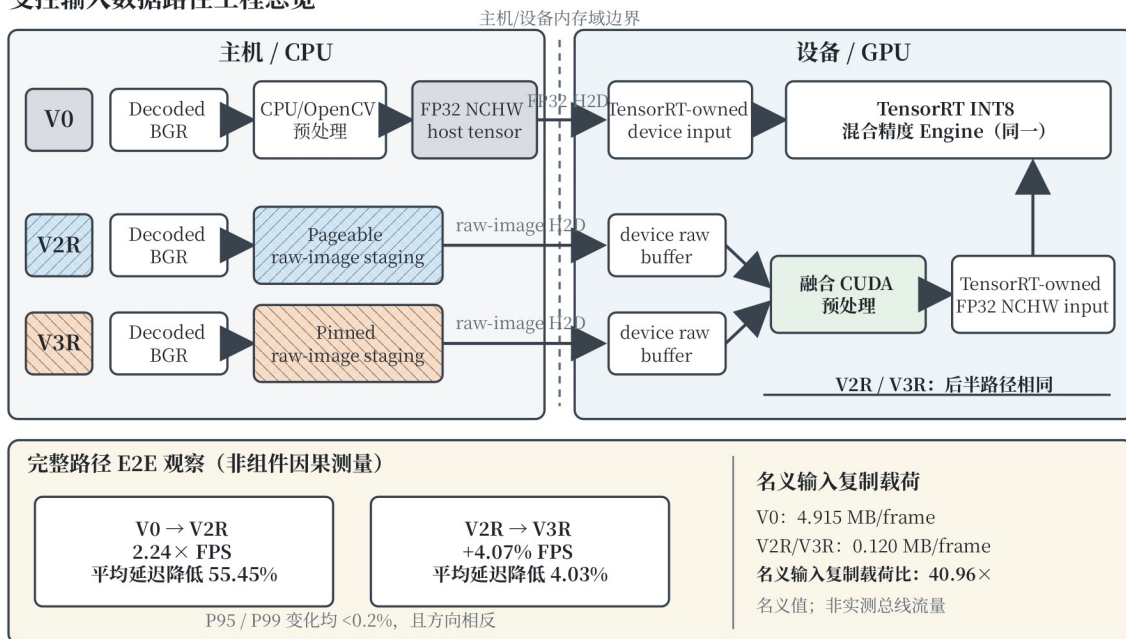


图1 V0、V2R 和 V3R 受控数据路径及完整路径观测。V0 在主机侧形成 FP32 NCHW 输入张量，V2R/V3R 将打包原始图像复制到设备并在 GPU 侧形成 TensorRT 输入；V3R 仅将 V2R 的 pageable 暂存替换为 pinned 暂存。性能数字表示完整端到端路径比较，不归因于单一组件。输入复制载荷为名义值，不等于实测总线流量。

全文首先定义模型、数据集、部署环境、和 V3R 的输入形成实现及正确性控制；继而给出统一实验协议；最后从正确性、整体 E2E 性能、

名义输入复制载荷、运行级分布、相关工作定位和适用边界六个方面讨论结果。

1 系统对象与问题定义

1.1 模型、数据集与部署环境

本文研究对象为部署于 NVIDIA Jetson Orin Nano Super 边缘平台的 YOLOv8n 工业缺陷检测模型。实际记录的软件环境为 L4T R36.5、CUDA 12.6 和 TensorRT 10.3，模型输入固定为 640×640、batch size 为 1。NVIDIA 官方资料显示，JetPack 6.2.2 所含 Jetson Linux 36.5、CUDA 12.6 和 TensorRT 10.3 与上述组件版本相对应；本文仍以实际记录的组件版本而非 JetPack meta-package 推断作为环境身份^[13]。

数据采用 NEU-DET 目标检测标注及去重后的 split-v2。原始数据库包含 6 类钢材表面缺陷和 1800 幅 200×200 图像；去重后训练集、验证集和测试集分别包含 1260、359 和 180 幅图像，测试集成员保持不变。后续部署评价固定使用同一组 180 幅测试图像。固定推理对象由既有 YOLOv8n 通过校准式后训练量化构建；量化部署需要同时评价计算实现与任务性能^[14]，因此本文只将该 Engine 作为数据路径研究的共同前提。

正式校准使用去重训练集的 1260 幅图像并排除测试集，采用 IInt8EntropyCalibrator2、batch size 1 和 640×640 输入。校准数据经过与生产 CPU Preprocessor 一致的 BGR letterbox、RGB/NCHW 与 FP32/255 预处理；构建同时启用 INT8 与 FP16，并在支持位置保持 FP32 I/O，故本文统一称其为 TensorRT INT8 混合精度 Engine。正式构建采用强制 cache miss，1260 个校准 batch 全部执行；校准完成后生成并归档 cache，未将既有 cache 复用为正式构建输入。

1.2 E2E 数据路径与受控变量

完整部署路径不仅包含固定输入张量进入 TensorRT Engine 后的网络计算，还包括图像获取与解码、主机暂存、输入形成、必要的移动、推理与同步、后处理和结果构造。Jetson 检测部署通常同样区分预处理、模型推理和后处理^[15]。本文用下式表示统一计时边界内各执行阶段共同构成 E2E 耗时：

$$T_{E2E} = \sum_{k=1}^m T_k.$$

该式只说明本文统计完整处理路径，而非仅包含 TensorRT 网络推理时间；各个 T_k 未被独立插桩或测量。

V0 将解码后的 BGR 图像在主机侧形成 640×640 FP32 RGB/NCHW 张量，再把该张量复制到 TensorRT 设备输入。V2R/V3R 先暂存 packed BGR uint8 原始图像，通过 cudaMemcpy2DAsync 执行二维 H2D 复制，写入设备原始图像缓冲区，再由一个融合 CUDA 核函数完成 resize、padding、BGR→RGB、归一化和布局转换，并直接写入 TensorRT 管理的 FP32 NCHW 设备输入。V2R 与 V3R 复用同一 CUDA stream、相同 CUDA 预处理和相同下游拓扑；两者之间唯一隔离变量是原始图像主机暂存的分配类型。

锁页内存可为满足条件的主机—设备异步传输与计算重叠提供支持，但它是有限资源，实际收益取决于具体实现^[16]。集成 CPU/GPU 和 GPU 内存分配研究也表明，内存策略的表现取决于平台、工作负载和访问特征^[17,18]。本文因此把 pageable→pinned 暂存作为独立变量，而不预设其收益。CUDA stream 与异步复制的通用语义参照 CUDA 官方文档^[19]；当前实现仍为单帧顺序路径，不使用跨帧传输/计算重叠。

V0→V2R 覆盖主机 FP32 张量形成到原始图像 H2D 与 GPU 输入形成的完整重构；V2R→V3R 仅替换主机暂存分配类型。完整检测系统与 GPU 预处理研究均提示，局部数据路径变化必须放回 E2E 执行边界评价^[20]。局部优化对 E2E 性能的实际贡献取决于其在完整执行路径中的占比及瓶颈位置，因此组件级变化不能直接等价为完整系统加速^[21]。

1.3 统一计时边界与研究问题

推理系统的延迟、吞吐率与尾延迟描述不同性能属性，平均延迟不能替代分布尾部^[22]。系统基准也需要针对具体场景分别定义延迟和吞吐率边界^[23]。本文采用固定离线回放协议，不采用 MLPerf 的查询生成、样本数或统计推断规则。

逐帧延迟采用统一的源读取至结果写出前（source-to-pre-sink）外部边界：计时从获取数据源帧之前开始，包含图像获取与解码、路径对应的主机暂存和输入形成、H2D、TensorRT 推理与同步、必要的 D2H、CPU 后处理/NMS 及结果对象构造，在结果序列化和写文件之前结束。

进程级 FPS 则以每个独立进程的 1080 个测量帧除以完整测量阶段的 wall time 得到，包含协议中统一的结果输出处理。两类指标采用不同边界，FPS 不由逐帧延迟取倒数获得。

基于上述定义，本文只回答两个研究问题。RQ1：在固定模型、Engine 和工作负载下，将主机 FP32 张量形成路径重构为原始图像 H2D 与 GPU 融合预处理，对完整 E2E 性能产生多大影响？

RQ2：在 GPU 预处理、CUDA stream 和下游拓扑保持不变时，将 pageable 原始图像暂存替换为 pinned 暂存，是否进一步改善平均性能，以及 P95/P99 是否呈现一致的尾延迟改善？

2 数据路径工程方法

三条受控路径的主要配置及比较变量见表 1。

表 1 V0、V2R 和 V3R 受控数据路径的特征矩阵。三条路径使用相同检测器和 TensorRT Engine；V0 在主机侧形成 FP32 输入张量，V2R/V3R 在设备侧形成输入张量，且后两者仅在 pageable 与 pinned 原始图像暂存类型上不同。三条路径均为单帧顺序执行，无跨帧流水线。

路径特征	V0	V2R	V3R
Detector / Engine	相同	相同	相同
CPU 像素预处理	是	否	否
CUDA 预处理	否	是	是
主机 FP32 输入张量	是	否	否
打包原始图像暂存	否	Pageable	Pinned
原始图像 H2D	否	是	是
张量形成位置	主机	设备	设备
直接形成 TRT 设备输入	否	是	是
复用 TRT CUDA stream	—	是	是
跨帧流水线	否	否	否

2.1 V0：主机侧 FP32 张量形成

V0 首先将数据源图像解码为主机侧 CV_8UC3 BGR 图像。CPU/OpenCV 随后执行 640×640 letterbox：使用 INTER_LINEAR 完成 resize，以常数 114 填充，并依次进行 BGR→RGB、HWC→CHW 和 1/255 归一化，最终在主机侧形成 FP32 NCHW 张量。该 1×3×640×640 张量再通过 TensorRT 后端复制到设备输入缓冲区，Engine 完成推理后将输出复制回主机并执行检测框解码、置信度筛选和 NMS。

V0 的关键系统属性不是笼统的“CPU 预处理”，而是主机侧 FP32 张量形成：像素变换、布局转换和浮点张量构造均在主机完成，H2D 输入对象也是完整 FP32 NCHW 张量。该路径保留既有 OpenCV 语义，作为后续输入形成位置和主机侧数据表示重构的受控基线。

2.2 V2R：pageable 暂存与 GPU 输入形成

V2R 保留主机侧图像读取与解码，并将解码后的 CV_8UC3 BGR 图像逐行复制到可复用的 std::vector<uint8_t>，形成连续 packed BGR 原始图像暂存。对冻结工作负载中的 200×200 图像，暂存区的有效行宽为 600 B。Letterbox 几何参数

仍由主机按照统一模型合同计算，不在 GPU 端重新决定缩放比例或 padding 位置。

packed BGR 原始图像通过 cudaMemcpy2DAsync 执行二维 H2D 复制，写入持久化设备原始图像缓冲区。随后，一个融合 CUDA 预处理核函数在同一次 launch 内完成 resize、padding、BGR→RGB、1/255 归一化和 HWC→NCHW 布局转换，并直接写入 TensorRT 管理的 FP32 NCHW 设备输入。该 resize 实现针对固定 Jetson 平台、CV_8UC3 输入和 OpenCV 4.5.4 INTER_LINEAR 行为建立对齐合同，不构成通用 OpenCV GPU 实现。

CUDA 预处理器在帧循环前建立，复用 TensorRT Engine 的 CUDA stream 及其持久化设备输入缓冲区；TensorRT 推理沿用同一 CUDA stream 和单一 execution context。V2R 由此把 V0 的主机 FP32 张量形成与 FP32 H2D 路径，重构为原始图像暂存、通过 cudaMemcpy2DAsync 执行二维 H2D 复制以及设备侧输入形成。该实现是输入数据路径工程，不引入新的检测、量化或 TensorRT 算法。

V2R/V3R 的主机—设备内存域、缓冲区生命周期与单流执行语义见图 2。

GPU 路径实现与内存域

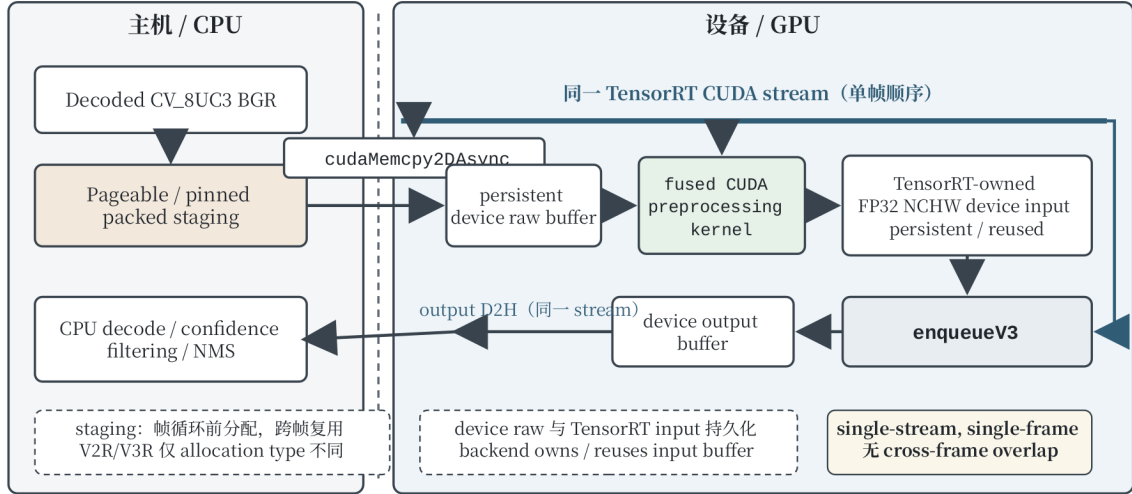


图2 V2R/V3R 的主机—设备内存域、缓冲区生命周期与单流执行语义。两条路径仅在主机侧 pageable/pinned 暂存类型上不同；cudaMemcpy2DAsync、融合 CUDA 预处理、enqueueV3 及输出 D2H 沿同一 TensorRT CUDA stream 顺序执行，暂存区、设备原始图像缓冲区和后端输入缓冲区跨帧复用。图中不表示跨帧重叠或流水线。

2.3 V3R：pinned 原始图像暂存隔离变量

V3R 与 V2R 共享相同的 packed BGR 语义、cudaMemcpy2DAsync、融合 CUDA 预处理核函数、TensorRT CUDA stream、Engine、后处理和下游执行拓扑。差异仅在于主机侧原始图像暂存：V3R 在帧循环前通过 cudaHostAlloc(..., cudaHostAllocDefault) 分配一个长生命周期锁页缓冲区，在全部帧之间复用，并在运行器生命周期结束时通过 cudaFreeHost 释放。每帧仍按行复制 width×3 字节，不发生逐帧锁页内存分配，也不存在失败后静默回退到 pageable 暂存。

因此，V3R 相对 V2R 的唯一隔离变量为原始图像主机暂存的分配类型。该受控关系使 pageable 与 pinned 暂存的观测差异不与第二套 CUDA 预处理、不同 Engine 或并发拓扑混合。既有嵌入式异构 SoC 研究同样说明内存与运行配置效果具有平台和负载依赖性^[24]，本文据此只评价当前固定实现中的边际变化。

三条正式路径均不使用 zero-copy、double buffering、multi-stream、跨帧流水线、显式传输/计算重叠或 GPU NMS。上述共同排除项只用于界定当前单帧顺序实现，不用于推断未测机制的性能。

2.4 正确性与生命周期控制

三条路径使用同一 180 幅测试图像、模型、Engine、输入尺寸、置信度阈值 0.25、IoU 阈值 0.45、max_nms=30000、max_det=300 和 class-

aware 单标签后处理。V0 提供既有任务正确性基线；V2R 沿用正式任务级评价，指标包括 precision、recall、mAP50、mAP50-95 以及类别级 AP50 和 recall。总体 mAP50-95、mAP50、precision 和 recall 的允许绝对差异分别为 0.005、0.005、0.010 和 0.010，类别级最大 AP50 和 recall 差异阈值分别为 0.020 和 0.030。

V3R 的任务指标来自对冻结预测结果的确定性统一评价，而非新的推理或第二次参数选择。该评价与 V0/V2R 使用相同的冻结工作负载和评价程序；V2R/V3R 之间的张量摘要校验值、检测结果摘要校验值、帧顺序、处理数量、丢帧计数和 EOS 状态进一步验证了唯一隔离变量身份。由此，三条路径均给出任务级评价结果，同时生命周期证据继续约束 V3R 的实现身份。正确性相等只针对本文冻结工作负载及评价口径，不扩展为任意未来输入上的普适等价。

3 实验设计

3.1 实验平台与模型配置

实验在 NVIDIA Jetson Orin Nano Super 平台上完成。实际记录的软件环境为 L4T R36.5、CUDA 12.6.11 (runtime 12.6.68)、TensorRT 10.3.0.30 和 OpenCV 4.5.4。已有 Jetson 检测框架评估表明，框架与平台条件会影响部署性能，因此本文不进行跨研究的绝对 FPS 比较^[25]。检测模型固定为

YOLOv8n，输入尺寸为 640×640、batch size 为 1，推理对象为第 1 节所述 TensorRT INT8 混合精度 Engine。三条路径不重新构建或校准 Engine，也不改变模型、检测阈值和后处理配置。

实验将工作负载、正确性条件、计时边界和统计口径作为统一条件。该设计与表面缺陷基准研究强调明确数据和评价边界的取向一致^[26]，但不声称遵循其他基准的正式规范。平台、模型和协议见表 2。

表 2 平台、模型与统一基准协议。三条路径在相同 Jetson 平台、YOLOv8n、冻结 TensorRT INT8 混合精度 Engine、固定测试工作负载和统一预热/测量协议下执行；表内仅保留复现实验所需的紧凑条件。

项目	设置
平台	NVIDIA Jetson Orin Nano Engineering Reference Developer Kit Super
软件栈	L4T R36.5; CUDA 12.6; TensorRT 10.3; OpenCV 4.5.4
Detector / 输入 Engine	YOLOv8n; 640 × 640; batch 1 TensorRT INT8 混合精度 (INT8 + FP16 fallback); host input FP32
校准	1260 张去重训练图像; IInt8EntropyCalibrator2; batch 1; 排除 test split
工作负载	固定 180 张 test 图像
路径	V0 / V2R / V3R; 单帧顺序执行
计时协议	60 帧预热; 每进程 1080 帧; 每路径 5 个独立进程
正式计时	关闭 diagnostics 与 profiling

正式运行使用 MAXN_SUPER 电源模式 (nvpmode mode 2)，未调用 jetson_clocks。时钟频率没有独立归档，因而不作为固定频率条件。运行前温度约为 46.8–47.1 °C，运行后约为 48.7–49.6 °C；这些仅是非连续的前后观察，不用于判断持续温度、运行时降频或功耗状态。

3.2 数据集与统一运行协议

性能实验采用 split-v2 固定测试集的 180 幅图像。输入成员和顺序由同一 manifest 限定，每次独立运行先预热 60 帧，完成同步并重置测量窗口，再测量 1080 帧，即连续回放 6 个完整周期。每条路径执行 5 个独立进程，V0、V2R 和 V3R 各形成 5400 个逐帧延迟样本，三条路径合计 16200 个样本。5 次运行均为独立进程，不在不同路径之间构造逐次对应关系。

15 个进程按照运行前确定的交错顺序执行：V0–V2R–V3R、V3R–V2R–V0、V2R–V0–V3R、V0–V3R–V2R 和 V2R–V3R–V0。全部 15 个进程均处理 1080 个测量帧、丢帧为 0，并通过

输入顺序、EOS 和生命周期准入。内部逐帧诊断计时与 profiling 保持关闭，避免不同路径采用不同测量机制。

3.3 正确性与性能指标

正确性评价采用冻结的 180 幅测试集和统一评价程序。V0/V2R 沿用既有正式任务评价结果，包括 Precision、Recall、mAP50、mAP50-95 及类别级 AP50/Recall；V3R 通过同一评价程序对冻结预测结果进行确定性评价，并结合 V2R/V3R 张量与检测结果摘要校验值及生命周期身份检查。V3R 评价不是新的推理或第二次参数选择环节。

性能指标包括进程级 FPS、逐帧平均延迟以及 P95、P99。对第 i 个独立运行，FPS 定义为

$$f_i = \frac{N}{T_i},$$

其中 $N=1080$ ， T_i 为该进程测量阶段的 wall time (s)。每条路径的平均 FPS 由 5 个进程级 FPS 算术平均得到：

$$\mu_f = \frac{1}{5} \sum_{i=1}^5 f_i.$$

FPS 离散程度以这 5 个值的样本标准差表示，采用样本均值 μ_f 并以 $5-1=4$ 为方差分母。该标准差是运行级描述量，不是置信区间。逐帧延迟采用 source-to-pre-sink 外部边界，每条路径合并 5 次运行的 5400 个样本后计算均值、P95 和 P99。百分位采用 Type-7 线性插值。设升序样本为 $x_{(1)}, \dots, x_{(n)}$ ，对于概率 p ，令

$$h = 1 + (n - 1)p, j = \lfloor h \rfloor, \gamma = h - j,$$

则内部位置的百分位数为

$$Q_p = (1 - \gamma)x_{(j)} + \gamma x_{(j+1)}.$$

分别取 $p=0.95$ 和 $p=0.99$ 得到 P95 与 P99。本文只报告上述固定工作负载下的描述性统计，不进行置信区间、假设检验或统计显著性推断。

4 结果与分析

4.1 正确性

表 3 给出 V0、V2R 和 V3R 的任务级正确性结果。在冻结的 180 幅测试集和统一评价程序下，三条路径的 Precision、Recall、mAP50 和 mAP50-95 分别为 0.6913、0.6991、0.6476 和 0.3523；类别级最大 AP50 和 Recall 路径间差异也均为 0。

表3 V0、V2R 和 V3R 的任务级正确性。

Precision、Recall、mAP50 和 mAP50-95 均由冻结预测证据按统一评估口径获得；各路径的汇总指标一致，类别级 AP50 与 Recall 的最大路径间差异均为 0。

	P	Pre	Rec	mA	mA
ath	cision	all	P50	P50-95	
0	V	0.6	0.6	0.6	0.3
		913	991	476	523
2R	V	0.6	0.6	0.6	0.3
		913	991	476	523
3R	V	0.6	0.6	0.6	0.3
		913	991	476	523

V3R 冻结预测结果经同一评价口径得到 Precision 0.6913、Recall 0.6991、mAP50 0.6476 和 mAP50-95 0.3523；相对 V0 和 V2R 的类别级最大 AP50 与 Recall 差异均为 0。由此，V0、V2R 和 V3R 在该冻结工作负载上获得一致的任务级评价结果，说明本轮输入数据路径重构未改变该测试集上的检测评价。V3R 结果来自冻结预测结果的确定性分析，而非新的推理或第二次参数选择；上述一致性不扩展为数学等价、逐位一致或未来输入上的普适结论。

4.2 整体 E2E 性能

V0→V2R 形成主要 E2E 性能增量，而 V2R→V3R 仅进一步改善平均性能。V0 的平均 FPS 为 54.600，平均延迟为 18.273 ms；V2R 分别为 122.122 FPS 和 8.140 ms。V2R 相对 V0 达到 2.24× FPS，平均延迟降低 55.45%。

V3R 的平均 FPS 为 127.097，平均延迟为 7.812 ms。相对 V2R，其 FPS 变化为+4.07%，平均延迟变化为-4.03%。两级结果表明，主要平均性能变化对应从主机侧 FP32 张量形成到原始图像 H2D 与 GPU 侧输入形成的完整重构；pageable→pinned 暂存是在该路径基础上的有限边际增量。

图 3 统一给出三条路径的平均 FPS、平均 E2E 延迟以及绝对 P95/P99。FPS 误差棒为 5 次独立进程级 FPS 的样本标准差，V0、V2R 和 V3R 的样本标准差分别为 0.223、0.492 和 1.279 FPS；延迟结果来自每条路径合并的 5400 个逐帧样本。

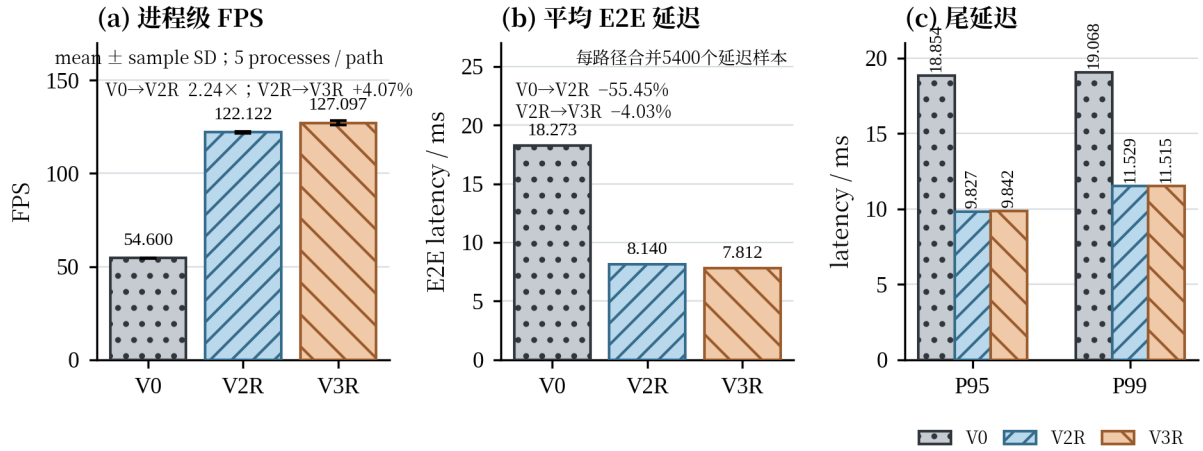


图3 三条受控路径的端到端性能。(a) 柱高为每条路径 5 个独立进程 FPS 的均值，误差棒为 5 个进程级 FPS 值的样本标准差；(b) 为每条路径合并 5400 个延迟样本得到的平均端到端延迟；(c) 为相同 5400 个合并延迟样本的 P95 和 P99。比较值描述完整路径差异，不构成对单一组件的因果归因。

4.3 数据路径分析

输入数据表示的变化可以用名义输入复制载荷 (nominal input-copy payload) 描述。V0 在主机端形成 1×3×640×640 的 FP32 NCHW 张量后复制至设备，其名义载荷为

$1 \times 3 \times 640 \times 640 \times 4 = 4,915,200 \text{ B} = 4.9152 \text{ MB/frame}$ 。

V2R/V3R 在冻结工作负载中复制 200×200 packed BGR uint8 图像，cudaMemcpy2DAsync 的

有效 copy width 为 600 B、height 为 200，名义载荷为

$600 \text{ B} \times 200 = 120,000 \text{ B} = 0.1200 \text{ MB/frame}$ 。

两类路径的名义输入复制载荷比为 40.96×。该确定性差异来自冻结图像几何和实现复制语义，表明输入形成位置与 H2D 数据表示发生了实质变化；它不等同于实际内存总线流量、带宽改善、H2D 时间降低或 40.96×传输加速，也不能单独

解释测得的 $2.24 \times$ E2E FPS 比。E2E 结果仍是包含全部计时边界阶段的完整路径观察。

4.4 运行级分布与尾延迟

图 4 给出运行级分布与尾延迟。5 次独立进程的描述性分布支持 V3R 平均性能优势具有运行级重复性，而不是由单一异常进程独自产生

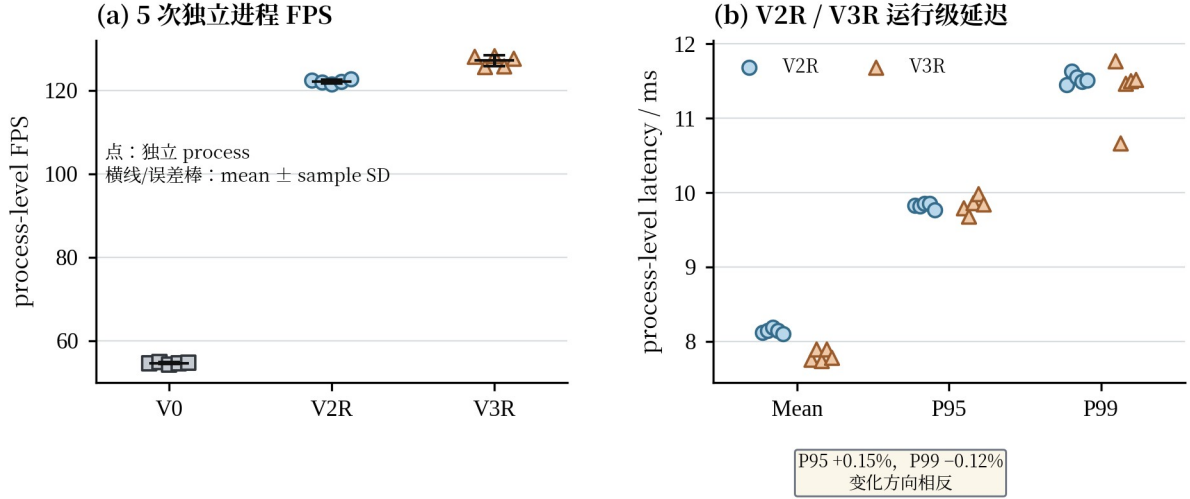


图 4 运行级分布与尾延迟。(a) 展示每条路径 5 个独立进程的 FPS 及描述性均值与样本标准差；(b) 展示 V2R/V3R 各进程的平均、P95 和 P99 延迟。各点为独立进程级描述量，横向偏移仅用于区分且不表示配对。正式合并样本 P95/P99 来自每条路径 5400 个延迟样本；P95 变化+0.15%、P99 变化-0.12%，方向相反，未形成一致的尾延迟改善证据。

尾延迟没有形成相同方向。V2R 的合并样本 P95 和 P99 分别为 9.827 和 11.529 ms，V3R 分别为 9.842 和 11.515 ms。V3R 相对 V2R 的 P95 为+0.15%（略慢），P99 为-0.12%（略快）。两项相对变化均低于 0.2%且方向相反，未形成一致的尾延迟改善证据。平均性能优势与尾部变化应分别解读，不把 pinned 暂存描述为稳定降低尾延迟的机制。

4.5 相关工作定位

表面缺陷研究通常以检测器结构和任务精度为中心，而边缘部署研究还涉及框架、预处理、内存管理和完整处理路径^[3,7,8,25]。本文与这些工作的区别在于固定 YOLOv8n 和同一 TensorRT INT8 混合精度 Engine，把主机侧 FP32 张量形成、GPU 侧融合预处理和 pageable/pinned 原始图像暂存组织成两级隔离比较，并同时报告统一工作负载下的任务正确性、E2E 延迟、进程级 FPS 和合并样本尾延迟行为。

这一定位不表示 GPU 预处理或 pinned 内存从未被研究，也不以不同论文之间的 FPS 绝对值

V2R 的进程级 FPS 范围为 121.443–122.759，V3R 为 125.595–128.301；对应的进程平均延迟范围分别为 8.098–8.185 ms 和 7.740–7.894 ms。两个范围在本次 5 次运行样本中没有重叠，但不同路径的进程不按运行次序建立逐次对应关系，本文也不据此进行显著性推断。

建立优劣结论。本文提供的是固定检测器/Engine 条件下的输入数据路径工程证据：V0→V2R 用于观察完整输入形成重构，V2R→V3R 用于观察主机暂存分配类型的边际变化。表 4 给出上述工作的研究属性定性比较，并区分“未报告”与“明确否”，避免将文献没有给出的实现信息推断为否定事实。

表 4 的判定口径为：边缘部署指在嵌入式或边缘设备上实际部署并报告实验结果；模型固定指所比较系统配置中的研究变量不包含检测网络结构、权重或模型参数变化；GPU 预处理指论文明确由 GPU/CUDA 执行模型推理前的图像预处理；host 内存策略指论文明确研究或配置主机侧内存分配、锁页/可分页或 managed 等策略；完整 E2E 指性能边界至少覆盖预处理、模型执行以及后处理或结果处理，而非仅网络推理；任务正确性指论文报告受比较部署或系统配置对应的任务级检测正确性指标；尾延迟指论文报告 P95、P99 或其他明确的百分位尾延迟指标，而非仅平均值或最大值。

表 4 相关工作的研究属性定性比较。表中汇总所审阅工作明确报告的研究属性；“明确否”仅用于原文明确排除的情形，“未报告”不等同于“否”。该比较用于定性定位，不表示优越性、首次性或唯一性。

工作	边缘部署	模型固定	GPU 预处理	host 内存策略	完整 E2E	任务正确性	尾延迟
Kim et al. (2025)	是	是	未报告	未报告	是	未报告	未报告
PRESTO (2025)	明确否	是	是	未报告	未报告	未报告	未报告
Tang & Qian (2024)	是	是	明确否	未报告	是	是	未报告
Shin & Kim (2022)	是	是	未报告	未报告	未报告	是	未报告
Bateni et al. (2020)	是	是	明确否	是	明确否	未报告	未报告
本文	是	是	是	是	是	是	是

4.6 局限性

本文证据来自单一 Jetson Orin Nano Super、单一 YOLOv8n TensorRT INT8 混合精度 Engine、单一 NEU-DET 数据集及固定 180 幅图像工作负载。输入采用离线文件回放，未覆盖真实相机、长期在线流或跨平台/跨模型泛化。运行状态只有非连续的前后温度观察；没有连续运行状态监测、独立归档的时钟频率、功耗测量或稳定电源状态证据。

实验也没有阶段级因果计时分解。40.96×名义输入复制载荷由实现和冻结几何确定，但 H2D 时间、预处理阶段时间、实际总线流量与总 DRAM 流量均未测量。因此，完整 E2E 差异不能分配为单个核函数、复制或像素操作的独立因果贡献。最后，5 次独立进程只用于描述运行级分布，本文没有置信区间、假设检验或统计显著性推断。

5 结论

本文面向 Jetson 端 YOLOv8n TensorRT INT8 混合精度部署，重构了输入形成与主机—设备数据路径：由 CPU/OpenCV 在主机侧形成 FP32 NCHW 张量并复制至设备，转为 packed BGR 原始图像暂存、通过 cudaMemcpy2DAsync 执行二维 H2D 复制，并由融合 CUDA 预处理直接形成 TensorRT 设备输入。在冻结 180 幅测试集上保持任务级评价一致条件下，V2R 相对 V0 达到 2.24× FPS，平均延迟降低 55.45%，表明完整输入形成路径重构构成本文受测配置中的主要 E2E 性能增量。

在 GPU 预处理、CUDA stream、Engine 和下游拓扑不变时，V3R 只将 pageable 暂存替换为 pinned 暂存。其相对 V2R 的 FPS 变化为 +4.07%，平均延迟变化为 -4.03%；该平均优势在 5 次独立进程的描述性分布中可见，并非由单一进程独自产生。P95 为 +0.15%、P99 为 -0.12%，两项变化均低于 0.2%且方向相反，未形成一致的尾延迟改善证据。当前证据只支持固

定实现中的有限平均增量，不支持普适的 pinned 内存收益或一致尾延迟改善。

上述结论限于单一 Jetson 平台、检测器/Engine、数据集和离线回放工作负载，且没有连续运行状态监测、功耗测量、独立时钟频率记录或阶段级因果计时。后续工作可在保持统一正确性与计时边界的前提下，补充真实相机和长期运行评价，测量 H2D、GPU 预处理与实际内存流量，并在其他模型和边缘平台上检验数据路径重构的适用范围。

参考文献

- [1] Lv X, Duan F, Jiang J, 等. Deep Metallic Surface Defect Detection: The New Benchmark and Detection Network[J]. Sensors, 2020, 20 (6) : 1562. DOI:10.3390/s20061562.
- [2] Song K, Yan Y. A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects[J]. Applied Surface Science, 2013, 285: 858-864. DOI:10.1016/j.apsusc.2013.09.002.
- [3] Shao R, Zhou M, Li M, 等. TD-Net: tiny defect detection network for industrial products[J]. Complex & Intelligent Systems, 2024, 10: 3943-3954. DOI:10.1007/s40747-024-01362-x.
- [4] Chu Y, Yu X, Rong X. A Lightweight Strip Steel Surface Defect Detection Network Based on Improved YOLOv8[J]. Sensors, 2024, 24 (19) : 6495. DOI:10.3390/s24196495.
- [5] Zhang T, Pang H, Jiang C. GDM-YOLO: A Model for Steel Surface Defect Detection Based on YOLOv8s[J]. IEEE Access, 2024, 12: 148817-148825. DOI:10.1109/ACCESS.2024.3476908.
- [6] Ultralytics. Explore Ultralytics YOLOv8[EB/OL]. (2023) [2026-08-07]. <https://docs.ultralytics.com/models/yolov8/>.
- [7] Stäcker L, Fei J, Heidenreich P, 等. Deployment of Deep Neural Networks for Object Detection on Edge AI Devices With Runtime Optimization[C]//2021 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW). IEEE, 2021: 1015-1022. DOI:10.1109/ICCVW54120.2021.00118.
- [8] Lee J, Han D, Kim J. PRESTO: Hybrid CPU-GPU Preprocessing Framework for Video-based AI Inference System[C]//Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services. ACM, 2025: 735-740. DOI:10.1145/3711875.3736688.

- [9] Weiss E, Caplan S, Horn K, 等. Real-Time Defect Detection in Electronic Components during Assembly through Deep Learning[J]. Electronics, 2024, 13 (8) : 1551. DOI:10.3390/electronics13081551.
- [10] Jacob B, Kligys S, Chen B, 等. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2018: 2704-2713.
- [11] Nagel M, Amjad R A, Van Baalen M, 等. Up or Down? Adaptive Rounding for Post-Training Quantization[C]//Proceedings of the 37th International Conference on Machine Learning: 卷 119. PMLR, 2020: 7197-7206.
- [12] NVIDIA Corporation. TensorRT 10.3 Release Notes[EB/OL]. (2024) [2026-08-07]. <https://docs.nvidia.com/deeplearning/tensorrt/archives/tensorrt-1030/pdf/TensorRT-Release-Notes.pdf>.
- [13] NVIDIA Corporation. JetPack SDK[EB/OL]. [2026-08-07]. <https://developer.nvidia.com/embedded/jetpack-sdk-622>.
- [14] Kim N J, Lee J, Kim H. HyQ: Hardware-Friendly Post-Training Quantization for CNN-Transformer Hybrid Networks[C]//Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24. International Joint Conferences on Artificial Intelligence Organization, 2024: 4291-4299. DOI:10.24963/ijcai.2024/474.
- [15] Tang Y, Qian Y. High-speed railway track components inspection framework based on YOLOv8 with high-performance model deployment[J]. High-speed Railway, 2024, 2: 42-50. DOI:10.1016/j.hspr.2024.02.001.
- [16] NVIDIA Corporation. CUDA C++ Best Practices Guide: Release 12.6[EB/OL]. (2024) [2026-08-07]. <https://docs.nvidia.com/cuda/archive/12.6.0/cuda-c-best-practices-guide/index.html>.
- [17] Bateni S, Wang Z, Zhu Y, 等. Co-Optimizing Performance and Memory Footprint Via Integrated CPU/GPU Memory Management, an Implementation on Autonomous Driving Platform[C]//2020 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS). IEEE, 2020: 310-323. DOI:10.1109/RTAS48715.2020.00007.
- [18] Rodriguez M, Yarza I, Kosmidis L, 等. Analysis of GPU Memory Allocation Characteristics[C]//16th Workshop on Parallel Programming and Run-Time Management Techniques for Many-Core Architectures and 14th Workshop on Design Tools and Architectures for Multicore Embedded Computing Platforms (PARMA-DITAM 2025): 卷 127. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025: 1:1-1:15. DOI:10.4230/OASIs.PARMA-DITAM.2025.1.
- [19] NVIDIA Corporation. CUDA C++ Programming Guide: Release 12.6[EB/OL]. (2024) [2026-08-07]. <https://docs.nvidia.com/cuda/archive/12.6.0/cuda-c-programming-guide/index.html>.
- [20] Kim S, Kim C, Kim S. Improving Performance of Real-Time Object Detection in Edge Device Through Concurrent Multi-Frame Processing[J]. IEEE Access, 2025, 13: 1522-1533. DOI:10.1109/ACCESS.2024.3520240.
- [21] Hill M D, Marty M R. Amdahl's Law in the Multicore Era[J]. Computer, 2008, 41 (7) : 33-38. DOI:10.1109/MC.2008.209.
- [22] Dean J, Barroso L A. The Tail at Scale[J]. Communications of the ACM, 2013, 56 (2) : 74-80. DOI:10.1145/2408776.2408794.
- [23] Reddi V J, Cheng C, Kanter D, 等. MLPerf Inference Benchmark[C]//2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA). IEEE, 2020: 446-459. DOI:10.1109/ISCA45697.2020.00045.
- [24] Archet A, Ventrux N, Gac N, 等. Energy-Efficient Use of an Embedded Heterogeneous SoC for the Inference of CNNs[C]//2023 26th Euromicro Conference on Digital System Design (DSD). IEEE, 2023: 30-38. DOI:10.1109/DSD60849.2023.00015.
- [25] Shin D-J, Kim J-J. A Deep Learning Framework Performance Evaluation to Use YOLO in Nvidia Jetson Platform[J]. Applied Sciences, 2022, 12 (8) : 3734. DOI:10.3390/app12083734.
- [26] Lema D G, Sánchez-González L, Usamentiaga R, 等. Benchmarking deep learning models for surface defect detection: a reproducible and statistically-rigorous approach[J/OL]. Journal of Intelligent Manufacturing, 2025. DOI:10.1007/s10845-025-02672-8.